Technische Hochschule Nürnberg

Fakultät Informatik

Verbessertes Routing für Fahrradfahrer – Klassifizierung von Smartphone-Sensordaten zur Erkennung von Bodenbelag und -qualität

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Christian Gügel

Matrikelnummer 364 3206

Erstgutachter: Prof. Dr. Ulrich von Zadow

Zweitgutachter: Prof. Dr. Tobias Bocklet

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Hinweis: Diese Erklärung ist mit Originalunterschrift (nicht gescannt) in das Papierexemplar der Abschlussarbeit fest einzubinden. Eine Spiralbindung ist nicht zulässig. Das digitale Exemplar enthält einen Scan der Erklärung mit Unterschrift.

Prüfungsrechtliche Erklärung der/des Studierenden

Angaben des bzw. der Studierenden:

Name: Vorname: Matrikel-Nr.:

Fakultät: Studiengang:

Semester:

Titel der Abschlussarbeit:

Ich versichere, dass ich die Arbeit selbständig verfasst, nicht anderweitig für Prüfungszwecke vorgelegt, alle benutzten Quellen und Hilfsmittel angegeben sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe. Außerdem versichere ich, dass der Hauptteil des Papierexemplares und des digitalen Exemplares identisch sind. Das digitale Exemplar enthält, falls gefordert, lediglich weitere Anlagen.

Ort, Datum, Unterschrift Studierende/Studierender

Erklärung der/des Studierenden zur Veröffentlichung der vorstehend bezeichneten Abschlussarbeit

Die Entscheidung über die vollständige oder auszugsweise Veröffentlichung der Abschlussarbeit liegt grundsätzlich erst einmal allein in der Zuständigkeit der/des studentischen Verfasserin/Verfassers. Nach dem Urheberrechtsgesetz (UrhG) erwirbt die Verfasserin/der Verfasser einer Abschlussarbeit mit Anfertigung ihrer/seiner Arbeit das alleinige Urheberrecht und grundsätzlich auch die hieraus resultierenden Nutzungsrechte wie z.B. Erstveröffentlichung (§ 12 UrhG), Verbreitung (§ 17 UrhG), Vervielfältigung (§ 16 UrhG), Online-Nutzung usw., also alle Rechte, die die nicht-kommerzielle oder kommerzielle Verwertung betreffen.

Die Hochschule und deren Beschäftigte werden Abschlussarbeiten oder Teile davon nicht ohne Zustimmung der/des studentischen Verfasserin/Verfassers veröffentlichen, insbesondere nicht öffentlich zugänglich in die Bibliothek der Hochschule einstellen.

Hiermit ☐ genehmige ich, wenn und soweit keine entgegenstehenden Vereinbarungen mit Dritten getroffen worden sind,

☐ genehmige ich **nicht,**

dass die oben genannte Abschlussarbeit durch die Technische Hochschule Nürnberg Georg Simon Ohm, ggf. nach Ablauf einer mittels eines auf der Abschlussarbeit aufgebrachten Sperrvermerks kenntlich gemachten Sperrfrist

von Jahren (0 - 5 Jahren ab Datum der Abgabe der Arbeit),

der Öffentlichkeit zugänglich gemacht wird. Im Falle der Genehmigung erfolgt diese unwiderruflich; hierzu wird der Abschlussarbeit ein Exemplar im digitalisierten PDF-Format an die Betreuer übermittelt. Bestimmungen der jeweils geltenden Studien- und Prüfungsordnung über Art und Umfang der im Rahmen der Arbeit abzugebenden Exemplare und Materialien werden hierdurch nicht berührt.

Ort, Datum, Unterschrift Studierende/Studierender

Datenschutz: Die Antragstellung ist regelmäßig mit der Speicherung und Verarbeitung der von Ihnen mitgeteilten Daten durch die Technische Hochschule Nürnberg Georg Simon Ohm verbunden. Weitere Informationen zum Umgang der Technischen Hochschule Nürnberg mit Ihren personenbezogenen Daten sind unter nachfolgendem Link abrufbar: https://www.th-nuernberg.de/datenschutz/

Kurzdarstellung

In dieser Arbeit wird die Effektivität von Smartphone-Sensordaten als Basis für die Klassifizierung von Bodenbelägen für Fahrradfahrer untersucht. Zur Klassifizierung werden mehrere Machine Learning Modelle getestet und verglichen. Über mehrere Monate wurden mit einem Citybike und einem handelsüblichen Smartphone die benötigten Test- und Trainingsdaten für unterschiedliche Straßenbeläge aufgenommen. Dabei kam die Applikation Phyphox zum Einsatz, die direkten Zugriff auf die Sensoren des Smartphones zur Aufnahme ermöglicht. Die Daten vorher händisch in die Kategorien Asphalt, Kopfstein, und Feldweg unterteilt, die die häufigsten Kategorien von Fahrradweg in einer europäischen Stadt darstellen. Zusätzlich werden sie in automatisch in Obeflächen-Rauheitsklassen der Stufen A bis F durch einen modifizierten International Roughness Index (IRI) - Alghorithmus eingeteilt. Am Ende hat sich herausgestellt, dass zwei Kombinationsmodelle die besten Ergebnisse lieferten: Ein Modell bestehend aus einem in Reihe geschalteten Convolutional Neural Network (CNN) und einem Long-Short-Term-Memory-Network (LSTM), sowie ein Modell aus einer in Reihe geschaltetem CNN und einem Transformer-Network. Beide erreichten eine Accuracy von 0,91.

Inhaltsverzeichnis

1. Einleitung **1**
1.1. KI und das Fahrrad im Alltag 1
1.2. Zielsetzung 1

2. Stand der Technik **3**
2.1. Die verwendeten Machine Learning Verfahren 3
2.1.1. Logistische Regression 3
2.1.2. Convolutional Neural Network 5
2.1.3. Long Short Term Memory 7
2.2. Verwandte Abhandlungen zur Klassifikation von Straßenqualität 8

3. Methodik **13**
3.1. Theoretischer Hintergrund 13
3.1.1. Metriken im Machine Learning 13
3.1.2. International Roughness Index 16
3.2. Aufbau des Experiments 19
3.3. Aufnahme der Testdaten 19

4. Diskussion der Ergebnisse **27**
4.1. Vergleich der verwendeten ML-Modelle 27
4.1.1. CNN-LSTM 27

5. Outlook **33**

6. Summary **35**

A. Supplemental Information **37**

Abbildungsverzeichnis **39**

Tabellenverzeichnis **41**

List of Listings **43**

Literaturverzeichnis **45**

Glossar **49**

Kapitel 1.

Einleitung

1.1. KI und das Fahrrad im Alltag

Das Fahrrad ist aus dem heutigen Alltag nicht mehr wegzudenken, vor allem in der Stadt. In Zeiten der steigenden Benzinpreise, wachsenden Stadtbevölkerung und steigenden Unterhaltskosten sind die Menschen mehr denn je auf das Fahrrad angewiesen. Und in Zeiten von Smartphone und weitreichendem Internet steht den Radfahrenden eine große Auswahl an Navigationsprogrammen zur Verfügung, um ihr Ziel zu erreichen. Eine der bekanntesten Grundlagen für diese Systeme ist OpenStreetMap (OSM), eine kostenlose, Community-getriebener Weltkarte. Ihre Daten sind in vielen bekannnten Services wie Komoot, CycleStreets und OsmAnd [Open 26c].

Darüber hinaus basieren eine Bandbreite an Online-Routing-Services auf OSM-Daten, wie GraphHopper, OpenRouteService und OSRM [Open 26d]. Zur Bereitstellung und Kategorisierung dieser Daten nutzt OSM ein ausgefeiltes Tagging-System, das verschiedene Eigenschaften einer Straße oder eines Straßenabschnitts erfasst und in Kombination ein Gesamtbild dieser Straßen abgibt. Dabei können diese Tags objektive Gegebenheiten als auch subjektive Bewertungen darstellen. SSurface"gibt die Oberflächenart einer Straße wie Asphalt, Beton oder Pflasterstein an, eine objektive, da klar ersichtliche Kategorie [Open 26a]. Dagegen sind Qualitäten wie SSmoothness"deutlich weniger objektiv einzuschätzen. Die verschiedenen Abstufungen von ëxcellent"bis ïmpassableßtützen sich auf die Einschätzung der verschiedenen Beitragenden der OSM-Plattform. Als Orientierung dienen ihnen lange Beschreibungen der Abstufungen, die auch auf geschätzte Qualitäten der Straßen eingehen, sowie eine Einschätzung, für welche verschiedenen Fahrzeuge ein bestimmter Weg denn befahrbar wäre [Open 26b].

1.2. Zielsetzung

Das Ziel dieser Arbeit ist es, eine Grundlage für ein einfach zu erlernendes und gruppentaugliches Sammeln und objektiveres Klassifizieren von Daten zur Straßenqualität für Fahr-

radfahrer zu schaffen. Der Ansatz soll dabei die Art des Bodens sowie seine Qualität und Rauheit gleichermaßen klassifizieren und in Kategorien einteilen. Dabei soll das Aufnehmen von neuen Daten so einfach wie möglich gehalten werden und auch während regulärer Fahrt mit dem Fahrrad möglich sein. Das Modell soll dazu leicht um neue Daten und Features erweiterbar sein.

Kapitel 2.

Stand der Technik

2.1. Die verwendeten Machine Learning Verfahren

Zur Auswertung der Daten werden drei verschiedene Machine-Learning-Algorithmen entweder einzeln oder in Kombination verwendet: die lineare Regression, das CNN und das LSTM.

2.1.1. Logistische Regression

Im Gegensatz zur bekannteren linearen Regression bietet sich die logistische Regression deutlich mehr für die Zwecke dieser Arbeit an. Die lineare Regression ist darauf ausgelegt, quantitative Werte wie Verkaufszahlen vorauszusagen. Dazu sagt sie auch Werte außerhalb des Bereichs von 0 bis 1 voraus. Bei Straßenbelägen handelt es sich aber um qualitative Variablen. Diese stehen in keinem quantitativen Zusammenhang und sind unabhängig voneinander. Um mit diesen "Wertenärbeiten zu können, braucht man eine Methode, die nicht die Werte direkt, sondern Wahrscheinlichkeiten voraussagt, mit denen eine Beobachtung zu einem der Werte gehört.

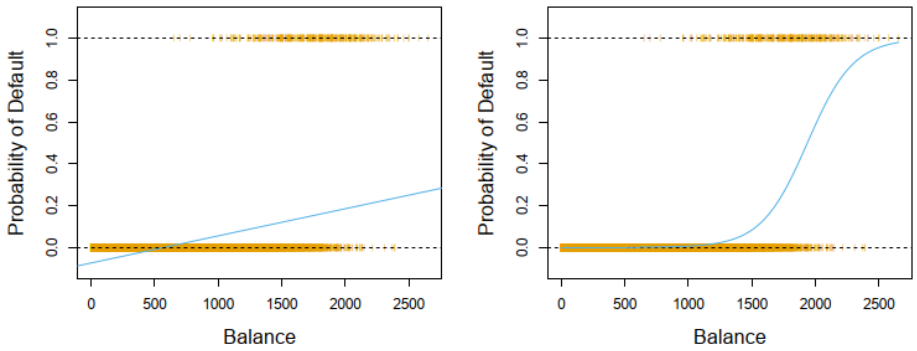

Abbildung 2.1.: Vergleich der geschätzten Wahrscheinlichkeiten für einen Testdatensatz zwischen linearer Regression (links) und logistischer Regression (rechts). [Jame 23]

Wie in 2.1 zu sehen, können bei der linearen Regression die vorausgesagten Wahrscheinlichkeiten sogar negativ werden. Auch wird der gesamte Bereich von 0 bis 1 nicht ausgenutzt. Die logistische Regression löst dieses Problem. Um die benötigten Werte zwischen 0 und 1 vorherzusagen, setzt die logistische Regression die "logistische Funktionëin:

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}} \tag{2.1}$$

Wie in Gleichung (2.1) zu sehen ist, nähert sich die Funktion für sehr große Werte von X der 1 an und für sehr kleine Werte der 0. Durch Umformung lässt sich die logistische Funktion als lineares Modell formulieren. Der Ausdruck auf der linken Seite wird als Log-Odds bezeichnet:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X \tag{2.2}$$

Wie in Gleichung (2.2) ersichtlich ist, stellt das logistische Regressionsmodell sicher, dass der Logit linear von den Eingabevariablen X abhängt. Hierbei sind β_0 und β_1 die Koeffizienten oder Gewichte, die mit den Trainingsdaten bestimmt werden müssen. Das X ist der Eingabewert (oder das Feature), dessen Zugehörigkeit bestimmt werden soll. Damit dies korrekt geschieht, wird die Maximum-Likelihood-Methode angewendet.

$$l(\beta_0, \beta_1) = \prod_{i:y_i=1} p(x_i) \prod_{i':y_{i'}=0} (1-p(x_{i'})) \tag{2.3}$$

Hierbei repräsentiert $p(x_i)$ die vorhergesagte Wahrscheinlichkeit für eine Beobachtung i, deren tatsächliche Klasse $y_i = 1$ ist, und $(1-p(x_{i'}))$ die Wahrscheinlichkeit für eine Beobachtung i', deren tatsächliche Klasse $y_{i'} = 0$ ist. Hat man die Koeffizienten erfolgreich bestimmt, muss man zum Klassifizieren nur noch den zu bestimmenden Wert in die X-Variable der logistischen Funktion einsetzen, um die Wahrscheinlichkeit zu erhalten, ob der Wert zu einer bestimmten Klasse gehört oder nicht.

Da in dieser Arbeit mit mehr als zwei Klassen gearbeitet wird, kommt die sogenannte multinomiale logistische Regression zum Einsatz. Es gibt hier verschiedene Ansätze, doch es läuft auf dasselbe Prinzip hinaus: Den einzelnen Klassen werden in ihrem Zusammenhang noch einmal einzelne Wahrscheinlichkeiten für ihren Eintritt zugewiesen. Dies geschieht entweder durch eine zufällig ausgewählte Klasse K als Basisklasse, mit der verglichen wird, oder komplett symmetrisch ohne eine einheitliche Referenz (die sogenannte Softmax-Methode).

Für die Methode mit K Klassen und einer Klasse als Referenzklasse (hier Klasse K) berechnet man die Log-Odds für jede der verbleibenden $K-1$ Klassen im Verhältnis zu dieser Referenzklasse:

$$\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right) = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p \tag{2.4}$$

Dabei beschreiben die Parameter $\beta_{k0}, \ldots, \beta_{kp}$ die Koeffizienten spezifisch für die Klasse k. Bei Softmax wird die Wahrscheinlichkeit für jede Klasse k direkt berechnet, ohne eine Referenzklasse zu definieren:

$$\Pr(Y=k|X=x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{\sum_{l=1}^{K} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}} \tag{2.5}$$

Die Funktion in Gleichung (2.5) stellt sicher, dass die berechneten Wahrscheinlichkeiten für alle K Klassen exakt auf 1 summieren. [1]

Da es sich bei der logistischen Regression um eine vergleichsweise simple und transparente mathematische Funktion handelt, ist sie sehr leichtgewichtig. Sie arbeitet schnell, verbraucht nicht viel Speicher und lässt sich leicht in viele verschiedene Endgeräte integrieren. Daher wird sie in dieser Arbeit als erste Möglichkeit eingesetzt.

2.1.2. Convolutional Neural Network

Das Neural Network ist das Standardwerkzeug im Bereich des Deep Learning.

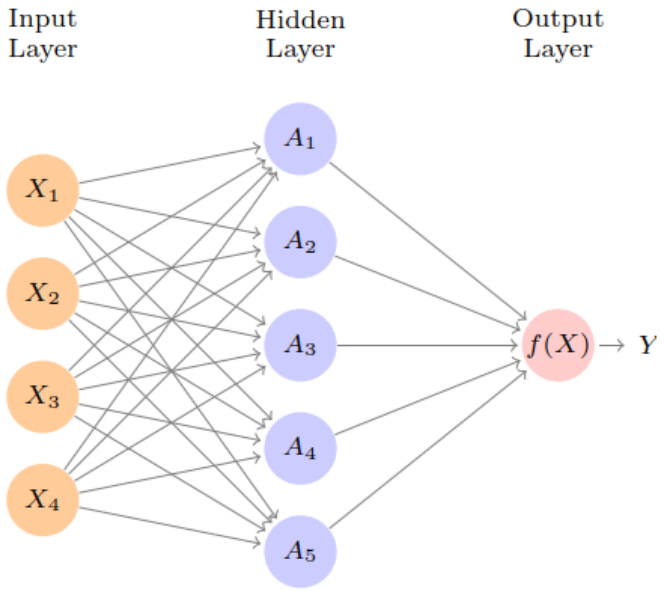

Abbildung 2.2.: Neural Network mit einem Hidden Layer. [Jame 23]

[1] [Jame 23]

In seinen Grundsätzen besteht es aus einem Input Layer, einem oder mehreren Hidden Layern und einem Output Layer. Wie in Abbildung 2.2 zu sehen ist, nimmt das Network eine Anzahl p an Inputs $X = (X_1, \ldots, X_p)$ und bildet im Hidden Layer eine nicht-lineare Transformation für die Kombination dieser Inputs in jedem Knoten A. Die Ergebnisse dieser sogenannten Aktivierungen werden an das Output Layer weitergereicht, um die nicht-lineare Funktion $f(x)$ zu bilden. Diese kann die Antwort Y voraussagen. Die Aktivierungsfunktionen in den K Knoten $A_k, k = 1, \ldots, K$ werden im Vorhinein festgelegt. Heutzutage verwendet man am häufigsten die ReLU-Funktion (Rectified Linear Unit):

$$g(z) = (z)_+ = \begin{cases} 0 & \text{falls } z < 0 \\ z & \text{sonst} \end{cases} \tag{2.6}$$

Diese ist einfacher zu berechnen und auch platzsparender als die vorher genutzte Sigmoid-Funktion. Das Modell aus Abbildung 2.2 beschreibt, wie aus den fünf Kombinationen von X fünf neue Features berechnet werden. Diese werden dann mit einer Aktivierungsfunktion transformiert. Am Ende ergibt sich ein lineares Modell zum Berechnen der Wahrscheinlichkeiten, ob ein Input zu einer Klasse gehört oder nicht.[2]

Die meisten neuronalen Netze haben jedoch eine Vielzahl an Hidden Layern, um komplexere Aufgaben zu lösen, und auch mehrere mögliche Outputs, um Daten einer Vielzahl an möglichen Klassen zuzuordnen. Hierzu wird wieder die Softmax-Funktion aus Gleichung (2.5) verwendet, um für jede mögliche Klasse die Wahrscheinlichkeit zu errechnen, um dann den Input zur Klasse mit der höchsten Wahrscheinlichkeit zuzuweisen.

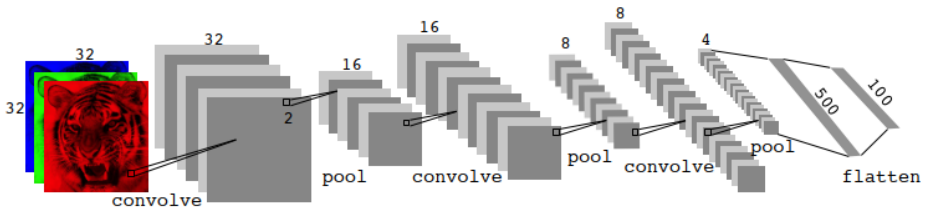

Abbildung 2.3.: Struktur eines tiefen CNNs mit einer Abfolge von Convolutional Layers und Pooling Layers. [Jame 23]

Das CNN zeichnet sich durch seine Eignung zur Klassifizierung mehrdimensionaler Daten aus. Anders als reguläre neuronale Netze werden die Hidden Layers hier in Convolutional Layers und Pooling Layers unterteilt. Diese Unterteilung ermöglicht ein hierarchisches Identifizieren von Features, von Low-Level nach High-Level, wobei mehrere Low-Level-Features zu High-Level-Features kombiniert werden, um schrittweise komplexere Muster zu erkennen. Die Convolutional Layers nutzen kleine Filtermatrizen (Kernels genannt), um schrittweise über die Daten zu gehen, die Werte, auf denen sie sich befinden, mit den Werten des Filters

[2] [Jame 23]

zu multiplizieren und die Produkte aufzusummieren. Je höher diese Summe, desto ähnlicher sind die originalen Daten dem Filter. Je niedriger, desto mehr unterscheiden sie sich. Während des Trainingsvorgangs passen sich die Filter an, um immer differenzierter lokale Muster zu erkennen und zu extrahieren. Als Ergebnis bekommt man eine Feature-Map, die die gefundenen Muster aufzeigt.

Das Pooling Layer fasst eine Feature-Map zu einem kleineren Bild zusammen. Mit dem Max-Pooling-Verfahren werden 2×2-Blöcke an Daten erfasst und auf den höchsten vorhandenen Wert reduziert. So bleiben alle wichtigen Daten erhalten. Diese Schritte werden in mehreren aufeinanderfolgenden Layern wiederholt, bis die Feature Map auf ein Minimum reduziert wurde. Die Map wird dann von ihrer mehrdimensionalen Struktur in eine eindimensionale Reihe an Daten überführt und in ein neuronales Netz gespeist, ähnlich wie oben beschrieben. Am Ende erhält man wieder eine Reihe an Wahrscheinlichkeitsfunktionen für die möglichen Klassen.[3]

Das CNN findet seine Anwendung meistens im Klassifizieren von Bildern und nicht in sequenziellen Datenreihen. Da beide aber in ihrer Struktur eine mehrdimensionale Abfolge an Daten darstellen, wird auch der Einsatz in diesem Fall untersucht.

2.1.3. Long Short Term Memory

Das LSTM gehört zur Gattung der Rekurrenten Neuronalen Netze (RNN) und ist darauf ausgelegt, einige fundementale Probleme die sowohl herkömmlichen CNNs als auch RNNs zu eigen sind zu lösen. Das RNN an sich setzt ein Konzept um, das den CNNs vorenthalten bleibt und theoretisch von hoher Bedeutung für diese Arbeit ist: das Behalten und Verwenden von vorher aufgenommenen Informationen zu einem späteren Zeitpung. Dies ermöglicht das Arbeiten mit sequentiellen Reihen an Daten, und das Verwenden früherer Zusammenhänge als Kontext für die Auswertung des Jetzigen, wie es sie in diesen Versuchen benötigt wird. Die Messreihe an Sensordaten stellt so eine Sequenz dar, und ist in ihrem Muster über einen Zeitraum aussagekräftiger als in den vereinzelten Ausschlägen ihrer Sensoren.

Doch das Problem mit herkömmlichen RNNs ist ihre Unfähigkeit Informationen über längere Zeitintervalle zu behalten. Josef Hochreiter hat dies in seiner 1997 erschienen Arbeit bereits thematisiert. In dieser spricht er vom sogenannten "decaying error back flow"(heute besser bekannt als das vanishing gradient problem), das bei zu langem Rückpropagieren der Fehler im Neuronalen Netz dazu führt, dass das Problem alsbald verschwindet und folglich die früheren Einheiten nichts mehr aus ihm lernen können. Analog hierzu ist das exploding gradient problem, bei dem der Fehler exponentiell ansteigt und so zu stark schwankenden Gewichten und unzuverlässigem Lernen führt. Um dieses Problem zu lösen, führt Hochreiter das "Constant Error Carouselëin. Dieses wird in eine "Memory Cell"mit Input und

[3] [Jame 23]

Output Gates integriert. So können Informationen über lange Zeiträume gespeichert und für einen konstanten Fehlerfluss gesort werden. Die Gates sorgen dafür, dass nur relevante Informationen hinein und hinausgelangen. [Hoch 97]

Moderne LSTMs lösen immer noch diese Probleme, haben jedoch einen abgewandelten Ansatz. In ihrer modernen Form (wie sie auch die in den Tests verwendete PyTorch Library implementiert) verfügen die Netze über sogenannte Forget Gates. Diese 1999 von Felix Gers beschriebenen Gates stellen sicher, dass die Memory Units des Netzes nicht fortwärend Daten ansammeln, die über die Zeit irrelevant werden. So werden Blöcke mit veralteten Daten resettet. [Gers 99]

Ein weiterer Forstschritt sind die ebenfalls von Gers erfundenen "Peephole Connections", die dem Netz ermöglichen präzise Zeitabstände zu messen und Rhytmen vorherzusagen. [Gers 00] Diese werden in dieser Arbeit, jedoch nicht verwendet.

2.2. Verwandte Abhandlungen zur Klassifikation von Straßenqualität

Im Review-Paper von Sattar et. al. werden verschiedene Studien ausgewertet und erfasst. Die Studien in diesem Paper handeln dabei hauptsächlich von Anomlie-Erfassungen wie Schlaglöcher, die die Fahrqualität und -sicherheit beeinflussen. Auch werden die Studien mit Autos und nicht mit Fahrrädern durchgeführt. [Satt 18]

Dennoch lassen sich aus den aufgezählten Methoden und ihren Ergebnissen eine Methodik für die Versuche dieser Arbeit ableiten. Das Review geht auf die Komplikationen beim Sammeln der Daten ein: Smartphones verfügen über unterschiedliche Sensoren, Fahrzeuge haben unterschiedliche Gewichte und Abmaße die ihr Schwingungsverhalten beeinflussen. Auch die Geschwindigkeit, mit der ein Fahrzeug fährt, beeinflusst die aufgenommenen vertikalen Beschleunigungen - es kommt zu unterschiedlichen Vibrationsmustern in den Aufnahmen. Als Sensoren für die Aufnahmen verwenden die verschiedenen Autoren die Acceleratoren ihrer Geräte, während GPS und Gyroskop-Sensoren als komplementäre Daten. Rotation und Schwerkraft werden als mögliche Datenquellen genannt. Im Preprocessing werden zwei Ansätze zur Behandlung der unterschiedlichen Anbringungswinkel der Smartphones genannt: Das Transformieren des Geräte-Koordinatensystems auf ein lokales zum einen, und die Verwendung der gesamten Ausschlagsstärke aller drei Sensorachsen für ein im Raum unabhängiges Signal zum anderen. Um Störfaktoren in den Signalen zu entfernen und die Daten zu glätten werden verschiedene Filter verwendet, darunter high-pass butterworth, moving-average und band-pass. Zum Processing der Daten listen Sattar et. al. verschiedene Methoden der Forscher auf. Am häufigsten kommen Schwellenwertmethoden und ML zum Einsatz.

Dem Problem mit der Geschwindigkeitsabhängigkeit der Signale wird ein Unterkapitel gewitmet. Einige Forscher normalisierten ihre Daten auf Referenzgeschwindigkeiten, andere teilten Geschwindigkeit in feste Kategorien ein, oder nutzen unterschiedliche Algorithmen je nach Geschwindigkeit. Keines dieser Methoden führte zu einer kompletten Lösung des Problems, sodass der Einfluss der Geschwindigkeit vollständig ausgeglichen wäre. [Satt 18]

Die Autoren des Reviews kommen zu dem Ergebnis, dass die Erkennungsmethoden nur in weitem Maße anwendbar sind, wenn ihre Daten aus einer Vielzahl von Zuständen stammen. Verschiedene Fahrzeuge (in diesem Fall Autos), verschiedene Smartphone-Modelle und Geschwindigkeiten werden empfohlen. Ebenso wird auf die Wichtigkeit einer korrekten Vorverarbeitung der Daten hervorgehoben. Die meisten Forscher verwenden starre Setups für ihre Smartphones. Die Autoren fordern eine variable Aufhängung der Geräte zu ermöglichen und die Differenzen per Transformation auszugleichen. [Satt 18]

Im Review stellte sich die Herangehensweise mit ML als verlässlicher heraus als die Schwellwert-Methoden. Es wird darauf hin-gewiesen, dass viele Autoren nur einen einzelnen Sensor ihres Smartphones für die Aufnahmen genutzt haben. Sattar et al. empfehlen eine Herangehensweise über Datenfusion von verschiedenen Sensoren, darunter Linear Acceleration (die Accelerometer-daten, aus denen die Schwerkraft bereits herausgerechnet wurde), Gyroskop und Schwerkraftsensor. Um das Problem mit den Diskrepanzen zwischen verschiedenen Test-Setups zu dämpfen wird Anwendung von Crowdsourcing empfohlen. Anwender konnten eine signifikante Steigerung ihrer Accuracy verzeichnen. In diesem Zusammenhang wird aber auch von den Ungenauigkeiten gewarnt, die von Positionsdaten herrüren, die von Mobilnetz und GPS bezogen werden. [Satt 18]

In ihrem Bericht gehen Seraj et al. näher auf ihre eigene Problematik mit Geschwindigkeitsverarbeitung ein. Sie verwenden ebenfalls Autos für ihre Aufnahmen und fokusieren sich auf Straßenanomalien, doch die Problematik bleibt auch für diese Arbeit relevant. Es wird darauf eingegangen, dass das GPS des Smartphones zur Verarbeitung der Geschwindigkeit verwendet wurde. Es stellte sich heraus, dass die erforderlichen Signale nicht immer erhältlich waren. Auch leiden GPS unter der sogenannten TTFF (time to first fix), die benötigt wird, um die Position über das Satellitensignal zu bestimmen. Im Falle komplett fehlender GPS Daten griffen die Autoren auf eine andere Tachnik zurück: die Hüllkurvendemodulation. Sie stellen fest, dass Komponenten wie Geschwindigkeit, Drift und Steigung im Signal der Accelerometer enthalten ist und können sie mit der Demodulation herausfiltern. [Sera 16]

Heidt und Dorer klassifizieren Fahrradstrecken nach drei verschiedenen Bodenbelägen - Asphalt, Pflasterstein und Kies zu je drei Qualitätsabstufungen. Hierfür benutzen sie ein industrielles Sensorgerät der Firma Bosch, das sie in Fest mit dem Rahmen des Fahrrads verbauen, um sekundäre Bewegungen zu unterbinden. Als Quellen für ihre Features verwenden sie Acceleratoren und Gyroskop des Geräts. Für ihre Deep Learning Modelle verwendeen sie die

rohen x-, y- und z-Werte der Daten, die lediglich genormt werden. Für die anderen Methoden werden aus den Rohdaten Arithmetisches Mittel, Standardabweichung, Maximal- und Minimalwert, mittlere Breite und Amplitude der Signalspitzen, mittlerer Spitzenabstand, mittlere positive und negative Steigung sowie maximale positive und minimale negative Steigung abgeleitet. Sie nehmen mit einer Sensorrate von bis zu 300 Hz auf und testen ihren Messreihen mehrere verschiedene Klassifikatoren. Zu ihnen gehören XGBoost, Random Forest, Support Vector Machine (SVM), CNN, LSTM, ein CNN-LSTM Modell, ein ResNet Modell und ein InceptionTime Modell. Für alle Modelle versuchen sie eine Vielzahl an Kombinationen der Hyperparameter, um die beste Accuracy beim Klassifizieren ihrer Testdaten zu erzielen. Mit optimierten Einstellungen erreichen die Residual Net Modelle eine Accuracy von 99%. Vor allem höhere Aufnahmefrequenzen um ca. 300Hz erweisen sich als besser. Bei den niedrigeren Frequenzen sind Signale rauer Oberflächen verschiedener Bodenklassen schwerer zu unterscheiden. Die auf diese Weise trainierten Modelle stellen sich jedoch als nicht transferierbar heraus. Beim Testem mit einem anderen Fahrrad erreichen die besten Modelle eine Accuracy von nur 26%. [Heid 21] Das von Heidt und Dorer beschriebene Vorgehen und die verwendeten Modelle dienen dieser Arbeit als Inspiration. Auch soll mit dem Verwenden einer handelsüblichen Handyhalterung mit einer loseren Verbindung zum Fahrrad ein transferierbareres Modell geschaffen werden. Das Paper von Heidt und Dorer weisen jedoch auf die große Herausforderung hin, die diese Transferierbarkeit darstellt. (Diskussion für Transferierbarkeit, Einstellungen für verschiedene Fahrräder, Aufnahmefrquenz)

In der Abhandlung von Massow et. al. wird komplett auf Machine Learning verzichtet. Das Ziel der Autoren ist es, eine allgemein nutzbare Methode zum Erfassen von Straßenqualität über Handysensoren während des Fahrradfahrens zu finden. Hierfür vergleichen sie verschiedene Berechnungsmethoden und führen in detaillierten Schritten die Vorverarbeitung der Daten vor. Als Berechnungsmethoden werden verglichen: den Pavement Condition Index (PCI), der über das Zählen von Anomalien in der Straße erhalten wird, der International Roughness Index (IRI) in verschiedenen Varianten, basierend auf dem quadratischen Mittelwert der Sensorbeschleunigungen, und die über die Fourier Transformation gewonnene Power Spectral Density (PSD) für Frequenzanalysen. Verglichen wird die Fähigkeit der verschiedenen Methoden, eine Teststrecke mit drei distinktiven Abschnitten - erst Pflastersteine, dann Asphalt, dann Gehweg - sichtbar in ihrer Qualität zu unterscheiden. In der Auswertung geht der IRI in einer Variante, in der er mit der durchschnittlichen Geschwindigkeit über eine festgelegte Distanz verrechnet wird als Sieger hervor, da er Straßen in Abstufung von Schlecht, Mittelgut und Gut verlässlich und in Abstufungen unterscheiden kann. Die genaue Art des Belags wird dabei nicht unterschieden. [Mass 24] Diese Arbeit baut auf den Ergebnissen und Herangehensweisen Massows und seines Teams auf.

Tarkiainen und Ikonen haben eine Studie zu Einflussfaktoren auf die Aufnahme von Smartphonesenoren beim Fahrradfahren durchgeführt. Wie auch in dieser Arbeit wird das Smarphone mit einer Halterung am Lenker montiert, um verschiedene Faktoren wie den Nei-

gungswinkel, die Aufnahmefrequenz und den Fahrradtyp zu testen. Dabei werden die Daten nicht klassifiziert, sondern statistisch und visuell in Matlab analysiert. Sie kommen zu dem Ergebnis, dass eine horizontale Aussrichtung des Smartphones auf dem Lenker, parallel zur Erdoberfläche, die besten Ergebnisse für Acceleratoren-Messungen hervorbringt. Das abgewinkelte Smartphone führt zu sichtbar gedämpften Ausschlägen der Acceleratoren im Vergleich zum horizontal montierten Smartphone. Ähnliche Effekte sind bei den Mittelwerten und der Standardabweichung der Acceleratoren zu sehen. Auch kommt das Paper zu dem Entschluss, dass eine niedrigere Aufnahmefrequenz weniger "Noiseïn die Daten bringt. Dies bietet Vorteil für die visuelle Erfassung, womit begründet wird, dass eine höhere Aufnahmefrequenz nicht unbedingt besser ist. Von Interesse ist auch die Berechnung der Acceleratorenfrequenz im vertikalen Bereich, die anders als bei Massow über eine Projektion des Accelerationsvektors in Richtung des Schwerkraftvektors erfolgt. [Tark 25]

Da das Paper sich mit der Analyse von Anamolien auf der Straße im Allgemeinen befasst und nicht um die Bestimmung verschiedener Beläge wird in dieser Arbeit eine höhere Aufnahmefrequenz gewählt, um feinere Unterschiede feststellen zu können. Auch wird das Smartphone in einem Winkel positioniert, was zwar für die Messungen suboptimal ist, aber eine deutlich natürlichere Position für den alltäglichen Gebrauch darstellt.

Kapitel 3.

Methodik

3.1. Theoretischer Hintergrund

3.1.1. Metriken im Machine Learning

Um die Güte eines trainierten ML-Modells zu bestimmen, stehen mehrere Kriterien zur Verfügung: Genauigkeit (Accuracy), Rate richtig positiver Ergebnisse (Recall), Rate falsch positiver Ergebnisse (FPR), Präzision (Precision) und der F1-Wert. Der Machine Learning Kurs von Google Developers gibt hierfür folgende Beschreibungen:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.1}$$

Formel 3.1 beschreibt den Anteil aller Klassifizierungen, die richtig waren. TP sind hierbei die True Positives (Wahr-Positiv), TN die True Negatives (Wahr-Negativ), FP die False Positives (Falsch-Positiv) und FN die False Negatives (Falsch-Negativ). Die Accuracy stellt ein generelles Maß der Güte von Modellen dar, hat aber Nachteile die zu beachten sind. Verfügt das Modell über unausgeglichene Datensets, in denen eine Klasse signifikant weniger vorkommt als andere, dann würde es immer noch eine sehr hohe Accuracy erreichen, obwohl diese seltene Klasse gar nicht korrekt klassifiziert werden kann. Hier müssen andere Metriken wie Recall und Precision herangezogen werden. [Goog 26]

Bei den gesammelten Straßendaten in diesem Projekt ist dies unweigerlich der Fall. Feld- und Waldwege kommen zwangsläufig selter vor als Asphalt und Kopfsteinpflaster, wenn man davon ausgeht, dass alle aufgenommenen Wege größtenteils einzigartig sind.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.2}$$

Formel 3.2 stellt den Anteil aller tatsächlich positiven Ergebnisse dar, die korrekt als positiv vorhergesagt wurden. Bei unausgewogenen Datensätzen und Modellen die in wenigen True Positives resultieren, ist der Recall ein aussagekräftigerer Wert als die Accuracy, da

eine falsche Einschätzung der zahlenmäßig schwächeren Klasse zu einer höheren Beeinträchtigung der Recall als der Accuracy führt. Recall wird vor allem verwendet, wenn negative Vorhersagen teurer sind als falsch Positive, z.B. bei Erkennung von Krankheiten. [Goog 26]

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.3}$$

In Formel 3.3 wird Anteil aller positiven Vorhersagen dargestellt, die tatsächlich positiv sind. Precision und Recall stehen in einem entgegengesetzten Verhältnis. Verringert sich der Recall-Wert, dann erhöht sich häufig die Precision. [Goog 26]

$$\text{F1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{3.4}$$

Die Formel 3.4 bildet den harmonischen Mittelwert von Precision und Recall ab. Er ist besonders für Datensets mit ungleichmäßiger Verteilung der Klassen geeignet. Liegen Precision und Recall nahe bei einander, ist der F1-Wert ihnen ähnlich. Gehen sie weit auseinander, liegt der F1-Wert näher am Schlechteren. Das macht ihn zu einem guten Prüfwert für bestimmte Datensätze. [Goog 26]

Um die trainierten ML-Modelle auszuwerten und zu vergleichen werden einige weitere Prinzipien genutzt. Die aufgenommenen Daten, die für die Modelle verwendet werden, werden in drei Sets gegliedert: Training, Validation und Test. Zur Anwendung kommt die in Chollet's Deep Learning with Python beschriebene K-fold cross-validation. Diese Technik wird benutzt, um die geringe Menge an Daten zu kompensieren und Varianz in den Prüfergebnissen zu vermeiden. Alle Daten werden gemisch und in K gleiche Partitionen unterteilt. Auf $K - 1$ der Partitionen wird trainiert, während die übrig gebliebene Partition zum Testen verwendet wird. Diese Prozedur wird K-mal wiederholt, sodass alle verfügbaren Daten zum Training verwendet werden. [Chol 25]

Um zu Messen wie weit die Vorhersagen der Modelle von den realen Gegebenheiten, den "Ground Truths"der Test-Daten entfernt sind, wird die sogenannte Loss-Function verwendet. Die Loss-Function nimmt die Vorhersagen und die wahren Klassen der Daten und berechnet den Distanzwert, um auszusagen, wie nah das Modell am eigentlichen Ziel für diese Dateninstanz liegt. Die Loss-Function ist fundamental für das Prinzip des Lernens im Deep Learning, da mit ihr die Gewichte des Modells so angepasst werden, dass der Distanzwert kontinuierlich verringert wird. [Chol 25]

Der Loss-Wert kann aber auch während des Trainings überwacht werden, sowohl für das Trainings-Set als auch für das Validation-Set. Mit diesen Entwicklungen lassen sich konkrete Aussagen über das Modell und seine Kapazitäten treffen. Vergleicht man die Loss-Werte

in Bezug auf die gerade aktiven Trainingsdaten mit den Loss-Werten gegenüber den Validierungsdaten über die vergangenen Epochen (konsekutive Trainingsdurchläufe über alle verfügbaren Trainingsdaten), so erhällt man z.B. folgendenen Graphen: [Chol 25]

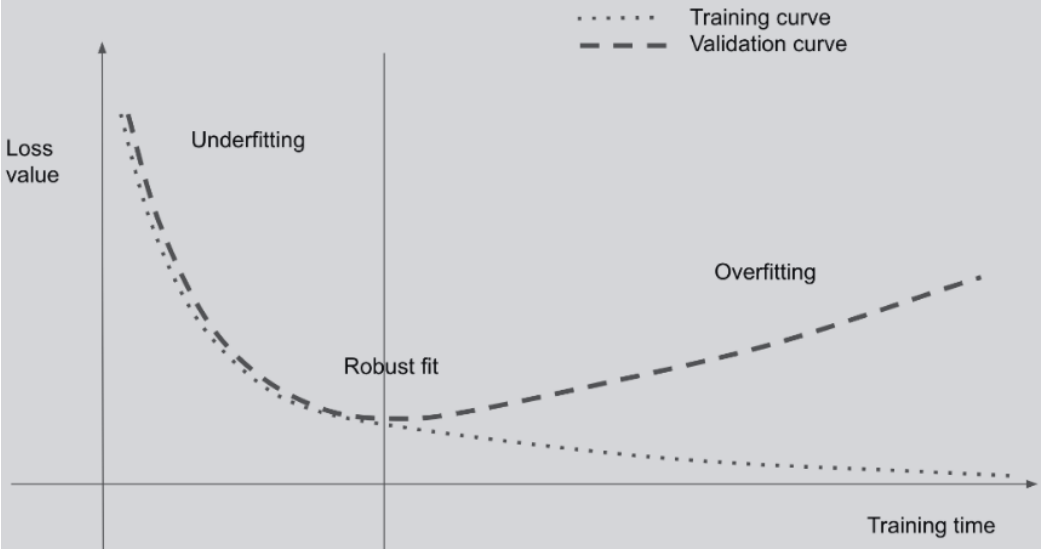

Abbildung 3.1.: Vergleich der Verläufe von Trainings-Loss und Validation-Loss. [Chol 25]

In Abbildung 3.1 wird ein idealisierter Verlauf der Loss-Werte mit fortschreitendem Training dargestellt. In den ersten Epochen werden die Gewichte des Modells noch weitläufiger eingestellt und der Loss für Trainings- und Validierungsdaten fällt stark ab. Dies geht so lange weiter, bis sich die Verläufe rapide trennen. Der Trainings-Loss fällt weiter, doch der Validierungs-Loss steigt an. Das Modell überfittet es erlernt Muster die überspizifisch für die Trainingsdaten sind und für ungesehene Daten keine Relevanz haben. Dadurch geht die Fähigkeit, neue Daten zu klassifizieren verloren, wie man am steigenden Validierungs-Loss erkennt. Das Modell optimiert seine Gewichte und Präzision weiter, aber es kann nicht mehr "generalisieren". [Chol 25]

Laut Chollet kommt es besonders häufig zu Overfitting, wenn die verwendeten Daten "verrauschtßind - also irrelevante, fehlerhafte und nicht interpretierbare Elemente enthalten. Auch stellen mehrdeutige Daten ein Problem dar. Klassen, deren Grenzen sehr subjektiv bewertet werden können und nur schwer in Grenzen zu fassen sind. Ein overfittetes Modell wird sich an diesen subjektivitäten aufhängen, was zu insgesamt schlechteren Klassifizierungen führt. Ebenso können selten vorkommende Features in den Daten zu falschen Korrelationen führen, prägnante Zusammenhänge die keine sind, aber trotzdem so vom Modell gelernt werden. [Chol 25]

Doch um zu ermittlen, ob eine gewählte Modellarchitektur auch für die Zwecke dieses Projekts geeignet ist, muss erst erwiesen werden, dass tatsächlich mit den verfügbaren Daten overfitten kann. Das Modell muss beweisen, dass es die Daten auswendig lernen kann. Wenn

dieser Beweis erbracht ist, drosselt man das Modell nach und nach, um eine möglichst optimale Generalsierung zu erhalten. Die finale Loss-Kurve sollte dann so aussehen: [Chol 25]

Darüberhinaus gibt es noch mehrere Hyperparameter, die vor dem Beginn des Trainings eines Deep Learning Modells beachtet werden müssen. Chollet beschreibt diese Parameter wiefolgt: Eine Batch ist eine Anzahl Samples aus dem gesamten Training-Set, die in einem Durchgang durch das neuronale Netz gefüttert werden. Auf diesen Samples findet ein Traininglauf statt, worauf im Anschluss die Gewichte des Netzes angepasst werden. Eine Epoche ist ein kompletter Durchlauf über alle verfügbaren Samples. Der Weight Decay ist gleichbedeutend mit der L2-Regularisierung, einer Methode um zu groß werdende Gewichte stärker zu bestrafen. Die learning Rate ist der Multiplikator, der bestimmt wie stark die Gewichte des neuronalen Netzes angepasst werden [Chol 25].

Wenn man eine geringere Batch-Größe hat, wird das Netz pro Epoche öfters und dazu auf weniger Daten trainiert. Eine höhere learning Rate würde größere Sprünge mit diesen niedrigeren Daten bedeuten, etc. Die Anpassung dieser Hyperparamter ist ein iterativer Prozess, der in Tandem mit der Überwachung der Learning Curve erfolgt. Hierbei wird die Generalsitation, also die Fähigkeit, neue und noch nie gesehene Daten richtig zu klassifizieren, priorisiert. Ein möglichst hoher F1-Wert wird hierfür angestrebt.

Für jedes Modell wird mindestens ein ermitteltes optimales Set an Hyperparamtern vorgestellt.

3.1.2. International Roughness Index

Die Bewertung von Oberflächenqualität in dieser Arbeit wird vom International Roughness Index (IRI) abgeleitet. Grundlage hierfür ist das mathematische Quarter-Car-Modell, welches die vertikale Reaktion eines Autoreifens und seiner direkten Fahrzeugkomponenten auf das Straßenprofil simuliert. Das Quarter-Car-Modell reduziert das Auto dabei auf ein einziges Rad sowie alle relevanten dynamischen Elemente: Federdämpfung, Reifenfederrate, Fahrwerksfeder, gefederte Masse und ungefederte Masse. Der IRI nach Sayers ist dabei die Akkumulation der simulierten Bewegungen zwischen gefederter und ungefederter Masse, die über eine Länge L normalisiert wird. [Saye 95]

$$IRI = \frac{1}{L} \int_0^{\frac{L}{V}} |\dot{z}_s - \dot{z}_u| \, dt \tag{3.5}$$

Hierbei ist L die Länge des gemessenen Straßenprofils, V die konstante Simulationsgeschwindigkeit (von Sayers als 80 km/h standatisiert), $\dot{z}_s$ die vertikale Geschwindigkeit der gefederten Masse (Sprung Mass) und $\dot{z}_u$ die vertikale Geschwindigkeit der ungefederten Masse (Unsprung Mass). [Saye 95]

Da diese Methode auf Autos ausgelegt ist und eine optimale Geschwindigkeit von 80 km/h unerreichbar ist, verwendet diese Arbeit stattdessen den Forschungsansatz von Massow et al. Dieser ist auf Fahrräder ausgelegt udn erfasst die direkte Reaktion des Fahrrads auf Unebenheit, und überträgt diese in einen abgeleiteten, auf die Fahrgeschwindigkeit ausgelegten IRI-Wert. [Mass 24]

$$IRI_{RMS+V} = \frac{n \cdot a_{z,\mathrm{RMS}}}{\sum_{i=1}^{n} v_i} \cdot 100 \tag{3.6}$$

Hierbei ist n die Anzahl der Messungen im Auswertungsfenster, v_i die am jeweiligen Messpunkt i aufgezeichnete Geschwindigkeit und $a_{z,\mathrm{RMS}}$ der Effektivwert der vertikalen Beschleunigung. [Mass 24]

$$a_{z,\mathrm{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} a_{z,i}^2} \tag{3.7}$$

Der Effektivwert ergibt sich aus Formel 3.7. Hier ist N die Anzahl der gemessenen linearen Beschleunigungswerte im Auswertungsfenster und a_z, i der i-te Messpunkt der vertikalen Beschleunigung.

$$a_{\mathrm{car}} = (1 - 0{,}88) \cdot a_{\mathrm{bike}} = 0{,}12 \cdot a_{\mathrm{bike}} \tag{3.8}$$

Um die Beschleunigungswerte für ihre IRI-Berechnung nutzbar zu machen, nehmen Massow et al. die in Gleichung 3.8 gezeigte Modifikation vor. Es wird erwähnt, dass ein Auto durch sein Aufhängungsystem auftretende Vibrationen um 88% verringern kann. Daher multiplizieren die Forschenden um Massow ihre gemessenen Beschleunigungswerte um den Kehrwert von 0,12. [Mass 24]

Diese Herangehensweise erweist sich in dieser Arbeit als nicht ausreichend, weswegen der Modifikator weiter angepasst und verringert werden muss. Für diese Arbeit wird für den berechneten IRI schließlich folgende vereinfachte Formel verwendet:

$$IRI_{RMS+V} = \frac{\sqrt{\frac{1}{N} \sum_{i=1}^{N} (k \cdot a_{z,i})^2}}{v} \cdot 100 \tag{3.9}$$

Hierbei ist k der Modifikator für die Beschleunigungswerte, $a_{z,i}$ die vertikalen linearen Beschleunigungswerte im Auswertungsfenster zum Zeitupunkt i und v die mittlere Fahrgeschwindigkeit im Auswertungsfenster. Zur Auswertung des IRI wird das von Massow et al. verwendete Auswertungsspektrum verwendet: [Mass 24]

- **Excellent:** IRI < 2
- **Good:** $2 \leq$ IRI < 4
- **Fair:** $4 \leq$ IRI < 7
- **Poor:** $7 \leq$ IRI < 10

- **Bad:** $10 \leq \text{IRI} < 18$
- **Failed:** $\text{IRI} \geq 18$

Hierbei muss erwähnt werden, dass die Kategorisierung selbst festgelegt ist. Andere Benutzer des IRI wie z.B. die World Bank geben als Beispiel für eine Einteilung u.a. folgende (m/km bezeichnet die gesamte vertikale Bewegung des Stoßdämpfers in Metern während eines gemessenen Kilometers):

- Very Good: $IRI < 2.5\,\text{m/km}$
- Good: $2.5 \leq IRI < 3.5\,\text{m/km}$
- Fair: $3.5 \leq IRI < 6.0\,\text{m/km}$
- Poor: $6.0 \leq IRI < 10.0\,\text{m/km}$
- Very Poor: $IRI \geq 10.0\,\text{m/km}$

Der Autor Archondo-Callao geht auch darauf ein, dass man nur vorsichtig pauschale Klassen wie good, fair etc. verwenden sollte. Bei der Roughness handelt es sich um nur einen Indikator für Straßenqualität, der andere Zustände übersehen kann. Aus sei eine einheitliche Klassengrenze über verschiedene Straßentypen hinweg sinnvoll, solange sie neutral benannt sind. So könne man unterschiedliche Straßentypen besser vergleichen, ohne inherent zu werten. [Arch 08]

Aus diesem Grund werden in dieser Arbeit zwar die Klassengrenzen von Massow et al. verwendet, das Namensschema wird aber neutraler gestaltet:

- **A:** $\text{IRI} < 2$
- **B:** $2 \leq \text{IRI} < 4$
- **C:** $4 \leq \text{IRI} < 7$
- **D:** $7 \leq \text{IRI} < 10$
- **E:** $10 \leq \text{IRI} < 18$
- **F:** $\text{IRI} \geq 18$

3.2. Aufbau des Experiments

Die Messreihen werden mit einem Citybike der Marke Pegasus Solero SL 7 aufgenommen. Das Fahrrad hat eine Rahmengröße von 280 mm und eine Reifengröße von 28 Zoll. Es wiegt ca. 18 kg. Mit dem Gewicht des Fahrers kommt man aufsummiert auf ca. 100 kg. Ein Smartphone der Marke Samsung Galaxy S23 kommt als Aufnahmegerät zum Einsatz. Es ist mit einer LSM6DSO Inertial Measurement Unit (LMU) ausgestattet, das Messung von Beschleunigung (Accelerometer) und Neigung (Gyroskop) in je 3 Achsen ermöglicht. [STMi 23]

Zugriff auf die Sensoren des Smartphones inklusive einstellbarer Aufnahmen von Messreihen wird durch die App "PhyPhox"der Universität RWTH Aachen ermöglicht. Das LMU erlaubt Aufnahmen bis zu einer Frequenz von 6.66 kHz. Dies ist für die Zwecke dieser Arbeit entschieden zu hoch. Aufbauend auf der Arbeit von Tarkiainen wurde sich für eine Aufnahmefrequenz von 150 Hz entschieden, um neben größeren Störungen wie Schlaglöchern und Bordsteinkanten auch granuläre Bodeneigenschaften erkennen zu können. Phyphox ermöglicht die simultane Aufnahme mehrerer Sensorarten. Für das Experiment werden der Accelerometer, das Gyroskop und das GPS des Smarphones als Daten verwendet. Hierbei wird darauf geachtet, dass die Option für das Verwenden purer Satteliten-Daten zur Standort- und Geschwindigkeitsbestimmung verwendet wird, um nicht verfälschende Signale von weniger präzisen Quellen wie WLAN oder Mobilfunk zusätzlich aufzunehmen.

Zusätzlich wird der Schwerkraft-Sensor während der Fahrt aufgenommen, um einen Ankerpunkt für die Projektion der Accelerometer-Daten zu gewährleisten.

Als Halterung für das Handy wird eine handelsübliche Lamicall-Halterung für Motorräder und Fahrräder verwendet. Das Smartphone wird in einem Winkel von ca. 45° angebracht, sodass das Smartphone vertikal in Fahrtrichtung zeigt.

3.3. Aufnahme der Testdaten

Über vier Monate hinweg werden Trainingsdaten von unterschiedlichen Straßentypen gesammelt. Der Fokus liegt auf den häufigsten Arten von Belägen, die in üblichen europäischen Städten anzutreffen sind:

- **Asphalt**
- **Verbund** (Betonstein- bzw. Verbundpflaster)
- **Pflastersteine** (Naturstein- bzw. Kopfsteinpflaster)
- **Feldweg** (Unbefestigter Weg, Erde, Schotter)

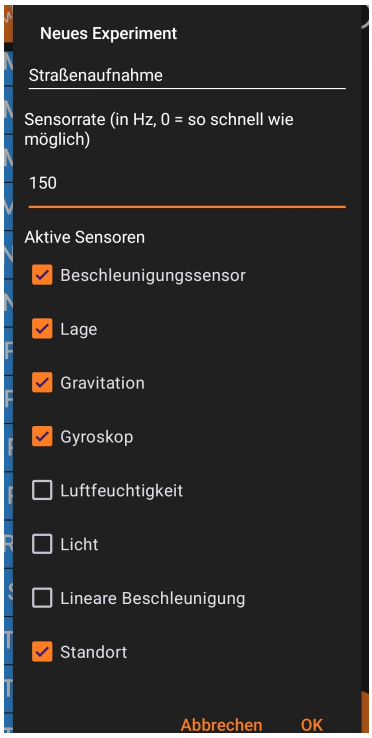

Abbildung 3.2.: Einstellungen für die Messungen in PhyPhox.

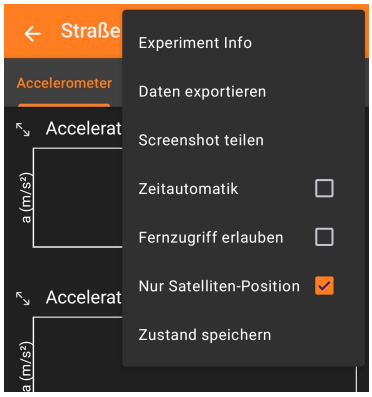

Abbildung 3.3.: Option für die Aufnahme purer GPS-Daten.

Abbildung 3.4.: Position der Halterung am Fahrrad.

Diese werden zusätzlich in Qualitätsstufen unterteilt, um zu testen, welche maximale Granularität bei der Klassifizierung möglich und brauchbar ist.

- **Asphalt**
  - **Gut:** Glatte Oberfläche, frisch asphaltiert, kaum messbare Erschütterungen.
  - **Mittel:** Raue Oberfläche, leichte Risse, ausgebesserte oder geflickte Stellen.
  - **Schlecht:** Größere Schäden, Risse, Schlaglöcher, Baumwurzeln, starke und abrupte vertikale Beschleunigungen.
- **Verbund**
  - **Gut:** Eben verlegte Steine, enge Fugen, geschlossene, flache Oberfläche.
  - **Mittel:** Leichte Setzungen im Untergrund, Fugen sind deutlich beim Fahren spürbar.
  - **Schlecht:** Starke Fugen, hochstehende Kanten oder fehlende Steine.
- **Pflastersteine**
  - **Gut:** Gerades historisches Pflaster, relativ flach, erzeugt ein konstantes Vibrationsmuster.
  - **Mittel:** Kopfsteinpflaster mit intakten Fugen. Erzeugt durchgehend starke Vibrationen.
  - **Schlecht:** Sehr grobes, unregelmäßiges Pflaster mit tiefen Fugen, lockeren Steinen oder tiefen Kuhlen. Erzeugt extreme Erschütterungen.
- **Feldweg**
  - **Gut:** Festgefahrener, trockener Erdboden oder sehr feiner, verdichteter Schotter.
  - **Mittel:** Loser Schotter, leichte Unebenheiten, Waschbrettmuster, flache Pfützen oder Spurrillen.
  - **Schlecht:** Grobe Steine, tiefe Matschlöcher, Wege in oder am Rand von Wäldern.
- **Stillstand** (Wenn Geschwindigkeit zu niedrig für erkennbare Datenmuster ist.)

Das Grundgerüst für Preprocessing und Datenverarbeitung für die verschiedenen ML-Modelle sind größenteils identisch und unterscheiden sich nur bedingt durch verschiedene Strukturen in den verwendeten Libraries. Deswegen wird hier nur exemplarisch auf eines der Prozesse eingegangen.

Das Projekt ist in Form eines Python-Notebooks aufgebaut. Im Folgenden wird der Aufbau schrittweise erklärt. Um die Reproduzierbarkeit der Ergebnisse sicherzustellen und die verfügbare Hardware effizient zu nutzen, wird das PyTorch-Modell zu Beginn initialisiert:

Listing 3.1: Initialisierung der Zufallsgeneratoren und Hardware-Zuweisung für NNs.

```
seed = 42
np.random.seed(seed)
random.seed(seed)
torch.manual_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

print(f"Device: {device}")
```

Der in Listing 3.1 dargestellte Codeausschnitt erfüllt drei wesentliche Funktionen für das Training der NNs:

- **Reproduzierbarkeit:** Durch das Festlegen eines Startwerts (*Seed*) für alle verwendeten Zufallsgeneratoren (`numpy`, `random` und `torch`) wird sichergestellt, dass Prozesse wie das Bestimmen der Gewichte bei jedem Durchlauf identisch sind.

- **Determinismus:** Die `cudnn`-Parameter zwingen die Grafikkarte dazu, nur deterministische Faltungs-Algorithmen zu verwenden. Dies verhindert Abweichungen, die durch dynamische Optimierungen der Hardware entstehen könnten.

- **Hardware-Zuweisung:** Der Code prüft automatisch die Hardware und weist das Modell der vorhandenen bestmöglichen Art zu, um das Training zu beschleunigen. Dabei werden Nvidia-GPUs (`cuda`) und Apple-Silicon-Chips (`mps`) unterstützt. Werden diese nicht gefunden, wird auf den Hauptprozessor (`cpu`) zurückgegriffen.

Die gesammelten Daten werden über ihre Dateipfade aufgerufen. Vorher wird eine Liste erstellt, in der sie eine festgelegte Straßengattung zugewiesen bekommen:

Listing 3.2: Zuweisung der Messfahrten zu den jeweiligen Straßenbelagsklassen (Ausschnitt).

```
class_map = {
    -1: "-1 Stillstand",
    0: "0 Asphalt / Verbund - gut",
    1: "1 Asphalt / Verbund - mittel",
    2: "2 Asphalt / Verbund - schlecht",
    3: "3 Kopfstein - gut",
    4: "4 Kopfstein - mittel",
    5: "5 Kopfstein - schlecht",
    9: "9 Feldweg - gut",
    10: "10 Feldweg - mittel",
    11: "11 Feldweg - schlecht",
}

all_files = {
    "Beispielstrasse_Mittel_28.12.25_14-11": class_map[1],
    "Beispielstrasse_Schotter_07.04.26_14-20": class_map[11],
    "Beispielstrasse_Kopfstein_alt_07.04.26_14-45": class_map[5],
}
```

Um die Sensordaten unabhängig von der individuellen Ausrichtung des Smartphones am Fahrrad auswerten zu können wird die vertikale Beschleunigung isoliert:

Listing 3.3: Berechnung der reinen vertikalen Beschleunigung durch Projektion auf den Schwerevektor.

```
def calculate_vertical_acceleration(acc_signals, gravity_signals):
    #1
    g_norms = np.linalg.norm(gravity_signals,
    axis=1, keepdims=True)

    #2
    g_norms[g_norms == 0] = 1.0

    #3
    g_hat = gravity_signals / g_norms

    #4
    a_v = np.sum(acc_signals * g_hat, axis=1)
```

```
14
15      return a_v.reshape(-1, 1)
```

Die in der Liste 3.3 gezeigte Funktion nutzt die von während der Fahrt aufgenommenen Gravitationsdaten. In Schritt 1 wird die Länge des Gravitationsvektors pro Zeitschritt errechnet. In Schritt 2 wird eine Division durch 0 verhindert. Schritt 3: Ein Numpy-Array an Gravitationsvektoren wird errechnet. Schritt 4 besteht aus einem Skalarprodukt der Accelerometerdaten mit ihren korrespondierenden Gravitationsvektoren (a_x mit g_x, a_y mit g_y, a_z mit g_z). Diese werden aufsummiert, um ein eindimensionales Array mit vertikalen Beschleunigungen zu erhalten.

Durch diese Projektion werden alle Störfaktoren wie Bremsen oder Lenken eliminiert. Nur die vertikalen Erschütterungen des Fahrrads bleiben übrig.

Um bessere Auskunft über die Qualität der Straße unabhängig ihres Typs zu erhalten wird ein approximierter IRI aus den Sensordaten berechnet:

Listing 3.4: Berechnung und Interpolation des fahrradbasierten IRI.

```
1  def calculate_IRI
2      (loc_interp, acc_vert_filtered, acc_time, az_modifier=0.035):
3      v = loc_interp[:, 0]
4      az = acc_vert_filtered[:, 0]
5
6      moving_mask = v > 0.5
7      v_moving = v[moving_mask]
8      az_moving = az[moving_mask]
9      time_moving = acc_time[moving_mask]
10
11     iri_results = []
12     points_per_block = 500
13
14     for i in range(0, len(az_moving)
15     - points_per_block, points_per_block):
16         group_az = az_moving[i : i + points_per_block]
17         group_v = v_moving[i : i + points_per_block]
18         group_time = time_moving[i]
19
20         az_car = group_az * az_modifier
21         az_rms = np.sqrt(np.mean(az_car**2))
22         mean_v = np.mean(group_v)
23
```

```
24            iri_val = (az_rms / mean_v) * 100 if mean_v > 0 else 0
25
26            iri_results.append({
27                'Time (s)': group_time,
28                'IRI': iri_val,
29                'az': az_rms,
30                'v': mean_v,
31                'len': len(group_az),
32            })
33
34        df_iri_final = pd.DataFrame(iri_results)
35
36        iri_interp_func = interp1d(
37            df_iri_final['Time (s)'].values,
38            df_iri_final['IRI'].values,
39            kind='linear',
40            bounds_error=False,
41            fill_value='extrapolate',
42        )
43
44        iri_interp = iri_interp_func(acc_time).reshape(-1, 1)
45        iri_interp[~moving_mask] = 0.0
46
47        return df_iri_final, iri_interp
```

Die in Listing 3.4 dargestellte Funktion verarbeitet die vertikale Beschleunigung und die Geschwindigkeit in drei Schritten:

1. **Datenfilterung (Masking):** Zunächst werden alle Messpunkte herausgefiltert, bei denen die Geschwindigkeit $v \leq 0{,}5\,\mathrm{m/s}$ beträgt. So wird sichergestellt, dass der errechnete IRI-Wert nicht durch zu niedrige Geschwindigkeit verzerrt wird.

2. **Segmentierung und Berechnung:** Das gefilterte Signal wird in Fenster von 500 Messpunkten unterteilt. Für jedes Fenster wird die vertikale Beschleunigung mit einem fahrradspezifischen Faktor (`az_modifier`) skaliert und deren quadratischer Mittelwert errechnet. Der lokale IRI ergibt sich anschließend aus dem Verhältnis des RMS-Wertes zur Durchschnittsgeschwindigkeit im jeweiligen Fenster.

3. **Interpolation:** Die ML-Modelle benötigen Feature-Vektoren in der ursprünglichen Abtastrate der Acceleratoren (dies wurde als Standard festgelegt, da es die wichtigs-

ten Werte sind). Die Ergebnisse werden linear auf die Zeitachse der Accelerometer-Messreihe interpoliert. Phasen des Stillstands erhalten den IRI-Wert null.

Kapitel 4.

Diskussion der Ergebnisse

4.1. Vergleich der verwendeten ML-Modelle

Die verwendeten Modelle wurden mit folgenden Daten trainiert:

Klasse (Label)	Anzahl Fenster	Dauer (Min.)	Dauer (Sek.)
-1 Stillstand	92	1.5	92.0
0 Asphalt / Verbund	2928	48.8	2928.0
1 Kopfsteinpflaster	2243	37.4	2243.0
2 Feldweg	1017	16.9	1017.0
Gesamt	**6280**	**104.7**	**6280.0**

Tabelle 4.1.: Verteilung der gesammelten Datenfenster bei einer Aufnahmefrequenz von 150 Hz

In der Aufzählung 4.1 wird die Gesamtzahl und Verteilung pro Klasse der verwendeten Datenfenster aufgezeigt. Die Aufnahmefrequenz dieser Datenreihen war 150 Hz, dazu wurde bei der Präparierung der Daten eine Fensterbreite von 450 Datenpunkten und eine Schrittweite von 150 Datenpunkten verwendet, um eine Überlappung von 1/3 zu erreichen.

Als vielversprechendeste Modell-Konfigurationen haben sich Kombinationsmodelle aus einem CNN zur anfänglichen Identifikation lokaler Merkmaler in den Datenreihen und ein nachgeschaltetes Modell zur Identifikation zeitlicher Zusammenhänge erwiesen. Sowohl ein LSTM als auch ein Transfomer-Modell kamen für letztere Aufgabe zum Einsatz. Die in Grafik 3.1 beschriebenen Learning Curves wurden zur Überwachung der Lernprozesse und fortwährenden Anpassung der Hyperparameter verwendet.

4.1.1. CNN-LSTM

Listing 4.1: PyTorch-Implementierung der hybriden CNN-LSTM Architektur

```
1 class CNNLSTMClassifier(nn.Module):
2     def __init__(self, output_dim, num_features):
```

```
        super(CNNLSTMClassifier, self).__init__()

        # CNN Block 1
        self.conv1 = nn.Conv1d(in_channels=num_features,
            out_channels=32, kernel_size=5, padding=2)
        self.bn1 = nn.BatchNorm1d(32)
        self.pool1 = nn.MaxPool1d(kernel_size=2)

        # CNN Block 2
        self.conv2 = nn.Conv1d(in_channels=32, out_channels=64,
            kernel_size=5, padding=2)
        self.bn2 = nn.BatchNorm1d(64)
        self.pool2 = nn.MaxPool1d(kernel_size=2)

        # LSTM Block
        self.lstm = nn.LSTM(input_size=64, hidden_size=128,
            num_layers=2, dropout=0.3, batch_first=True)

        # Classification Head
        self.dropout = nn.Dropout(0.5)
        self.out = nn.Linear(128, output_dim)

    def forward(self, x_ts):
        # Prepare for 1D Convolution: (Batch, Channels, Time)
        x_ts = x_ts.permute(0, 2, 1)

        # Apply CNN Layers
        x_ts = self.pool1(F.relu(self.bn1(self.conv1(x_ts))))
        x_ts = self.pool2(F.relu(self.bn2(self.conv2(x_ts))))

        # Prepare for LSTM: (Batch, Time, Channels)
        x_ts = x_ts.permute(0, 2, 1)

        # Apply LSTM
        lstm_out, _ = self.lstm(x_ts)

        # Global Average Pooling across time dimension
        # Shape: (Batch, 128)
        ts_features = torch.mean(lstm_out, dim=1)
        x = self.dropout(ts_features)
```

```
43          return self.out(x)
```

Das in 4.1 abgebildete finale CNN-LSTM-Hybridmodell besteht aus einem CNN mit zwei eindimensionalen Faltungsschichten. Diese Schichten bilden die eingehenden Datenkanäle erst auf 32 und dann auf 64 Feature-Maps. Danach folgt eine Batch-Normalisierung zur Stabilisierung des Trainingsprozesses und die ReLU-Aktivierungsfunktion. Es folgt eine Max-Pooling-Funktion mit Größe 2. Diese dient zur Feature-Verdichtung und Maxima-Bestimmung, wodurch wichtigere Features deutlicher hervortreten [Chol 25]. Die zweite Faltungsschicht wiederholt den Prozess und erweitert die Feature-Maps auf 64 Kanäle.

Anstatt der üblichen Global Pooling Schicht hat das Modell ein zweischichtiges LSTM, um zeitliche Zusammenhänge in den Daten zu lernen. Die verborgene Dimension ist auf 128 Neuronen festgelegt. Um Overfitting zwischen den beiden LSTM-Schichten zu minimieren, wird ein Dropout von 30 % verwendet. Diese Technik setzt eine zufällig ausgewählte Anzahl an Features während des Trainings auf 0, um zufällige, nicht-signifikante erlernte Muster aufzubrechen und so Generalisierung zu verbessern und Overfitting zu vermeiden. Chollet empfielt für RNNs eine Funktion namens Recurrent Dropout [Chol 25]. Diese ist in der PyTorch Bibliothek jedoch nicht verfügbar. Mit einem Global Average Pooling Layer wird der Mittelwert über alle übrigen Zeitschritte berechnet.

Zum Schluss wird noch eine Dropout-Rate von 50 % auf das gesamte Modell angewendet. Danach reduziert die Ausgabeschicht die Features auf die Anzahl der Straßenklassen.

Das Modell wird nach Ermittlung der optimalen Trainingsdauer auf allen Daten mit 90 Epochen trainiert. K-Fold Crossvalidation mit 5 Folds wird verwendet, um dem Ungleichgewicht in den vorhandenen Klassengattungen der Daten entgegenzuwirken. Zusätzlich werden die Datenreihen in Teilabschnitte von ca. 30 Sekunden eingeteilt, um die unterschiedlichen Längen der Datenreihen auszugleichen. Um Overfitting entgegenzuwirken werden verschiedene Schritte zur Regularisierung vorgenommen: Die Batch Size betrug 64 Samples, die Learning Rate betrug 0,00005 und der Weight Decay betrug 0,00001.

Eine weitere Hyperparameter-Kombination die zu ähnlichen Ergebnissen geführt hat war: Training für 120 Epochen, 5 Folds, Batch Size von 256, Learing Rate von 0,0001 und Weight Decay von 0,0002. Diese Parameter führen zu deutlich schnellerem Trainieren aufgrund der größeren Batch Size, aber zeigen eine leicht schlechtere Generalisierung auf. In den folgenden Darstellungen wird erstere Hyperparameter-Kombination verwendet.

In Tabelle 4.2 sind die Ergebnisse des Trainings der fünf Durchläufe zu sehen. Der Classification Report der sklearn Library wird herfür verwendet. Die 6280 Klassifizierungen decken sich mit den Input-Fenstern; jedes verfügbare Fenster wurde ein Mal klassifiziert. Hierbei handelt es sich nicht um den Durchschnitt der fünf Folds, sondern um alle gepoolten Vorhersagen der fünf Test-Sets. Macro Average bezeichnet hierbei den Wert, der den Durchschnitt

Klasse (Label)	Precision	Recall	F1-Score	Support
-1 Stillstand	0.736	1.000	0.848	92
0 Asphalt / Verbund	0.963	0.965	0.964	2928
1 Kopfsteinpflaster	0.939	0.943	0.941	2243
2 Feldweg	0.842	0.802	0.822	1017
Accuracy			**0.931**	6280
Macro Avg	0.870	0.928	0.894	6280
Weighted Avg	0.932	0.931	0.931	6280

Tabelle 4.2.: Klassifikationsergebnisse des CNN-LSTM Modells

der resultierenden Werte der oberen Spalte berechnet und somit alle Klassen als gleichgewichtet betrachtet. Hier treten Schwächen in weniger häufig auftretenden Klassen besonders hervor [Scik 26]. Dagegen berücksichtigt der Weighted Average die vorhandenen Samples der Klassen und gewichtet die Zahlen folglich als mehr oder weniger wichtig [Scik 26].

Wie zu sehen ist, erreicht das Modell einen Weighted Average von ca. 93 % bei Precision, Recall und F1. Darüber hinaus ist die Gesamt-Accuracy der fünf Folds ebenfalls bei 93,1 %. Der einzige Wert der heraussticht, ist die unzulängliche Precision bei der Klasse Stillstand. Diese Klasse ist jedoch so wenig vertreten, dass es den Weighted Average nicht beeinträchtigt. Es handelt sich hierbei wahrscheinlich um ein Problem mit dem Fine-Tuning der Data-Loader-Funktion, die angepasst werden muss, wenn es weiterhin zu Problemen kommt. Sonst sind die Ergebnisse insgesamt zufriedenstellend. Die Klasse Feldweg erreicht Werte von 84,2 % in Precision und 80,2 % in Recall, obwohl sie nur 16,2 % aller verfügbaren Daten ausmacht. Diese Diskrepanz sollte sich mit zusätzlichen Trainingsdaten ausgeglichen lassen.

Die folgende Globale Confusion Matrix visualisiert die Klassifikationsverteilung deutlicher:

Wie in Abbildung 4.1 zu sehen ist, finden die meisten fehlerhaften Klassifikationen zwischen den Klassen Kopfsteinpflaster und Feldweg statt.

Im Laufe der Untersuchungen haben sich Smartphonesensoren als Basis der Messreihen zur Klassifizierung von Bodenbelägen, sowie das gewählte Setup als vielversprechend und ausbaufähig erwiesen. Es wird jedoch noch weitere Arbeit und fortwährende Anpassungen sowohl im gewählten ML-Modell als auch in im Aufnahme-Setup notwendig sein.

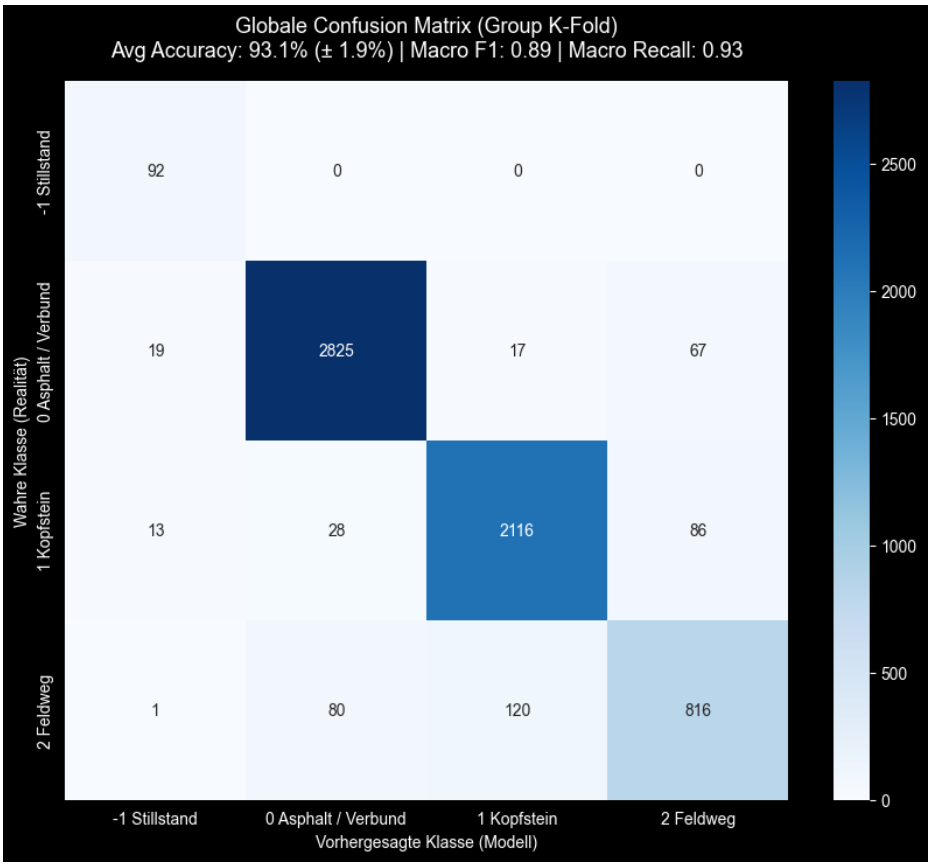

Abbildung 4.1.: Globale Confusion Matrix CNN-LSTM

Kapitel 5.

Outlook

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kapitel 6.

Summary

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Anhang A.

Supplemental Information

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Abbildungsverzeichnis

2.1. Vergleich der geschätzten Wahrscheinlichkeiten für einen Testdatensatz zwischen linearer Regression (links) und logistischer Regression (rechts). [Jame 23] 3
2.2. Neural Network mit einem Hidden Layer. [Jame 23] 5
2.3. Struktur eines tiefen CNNs mit einer Abfolge von Convolutional Layers und Pooling Layers. [Jame 23] 6

3.1. Vergleich der Verläufe von Trainings-Loss und Validation-Loss. [Chol 25] 15
3.2. Einstellungen für die Messungen in PhyPhox. 20
3.3. Option für die Aufnahme purer GPS-Daten. 20
3.4. Position der Halterung am Fahrrad. 20

4.1. Globale Confusion Matrix CNN-LSTM 31

Tabellenverzeichnis

4.1. Verteilung der gesammelten Datenfenster bei einer Aufnahmefrequenz von 150 Hz 27
4.2. Klassifikationsergebnisse des CNN-LSTM Modells 30

List of Listings

3.1. Initialisierung der Zufallsgeneratoren und Hardware-Zuweisung für NNs. 22
3.2. Zuweisung der Messfahrten zu den jeweiligen Straßenbelagsklassen (Ausschnitt). . 23
3.3. Berechnung der reinen vertikalen Beschleunigung durch Projektion auf den Schwerevektor. 23
3.4. Berechnung und Interpolation des fahrradbasierten IRI. 24

4.1. PyTorch-Implementierung der hybriden CNN-LSTM Architektur 27

Literaturverzeichnis

[Afen 20] A. Afenika, P. H. Gunawan, and D. Tarwidi. "Classification of Road Surface Quality Based on SVM Method". *Journal of Physics: Conference Series*, Vol. 1641, No. 1, p. 012064, nov 2020.

[Andr 21] Andreas. "GraphHopper Routing Engine 4.0 Released". https://www.graphhopper.com/blog/2021/09/29/graphhopper-routing-engine-4-0-released/, 2021. Besucht am: 12. Oktober 2025.

[Arch 08] R. Archondo-Callao. "Applying the HDM-4 Model to Strategic Planning of Road Works". World Bank Publications - Reports 17419, The World Bank Group, 2008.

[Chol 25] F. Chollet and M. Watson. *Deep Learning with Python*. Manning Publications Co. LLC, New York, 2025.

[Gers 00] F. Gers and J. Schmidhuber. "Recurrent nets that time and count". In: *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, pp. 189–194 vol.3, 2000.

[Gers 99] F. Gers, J. Schmidhuber, and F. Cummins. "Learning to forget: continual prediction with LSTM". In: *1999 Ninth International Conference on Artificial Neural Networks ICANN 99. (Conf. Publ. No. 470)*, pp. 850–855 vol.2, 1999.

[Goog 26] Google Developers. "Machine Learning Crash Course: Klassifizierung – Genauigkeit, Präzision und Trefferquote". https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=de, 2026. Besucht am: 10. April 2026.

[Heid 21] J. Heidt and K. Dorer. "Classification and Prediction of Bicycle-Road-Quality using IMU Data". *Artificial Intelligence - Application in Life Sciences and Beyond*, Vol. 2112.05657, 2021.

[Hoch 97] S. Hochreiter and J. Schmidhuber. "LONG SHORT-TERM MEMORY". *Neural Computation*, Vol. 9, No. 8, 1997.

[Hoff 13] M. Hoffmann, M. Mock, and M. May. "Road-quality Classification and Bump Detection with Bicycle-Mounted Smartphones.". In: *UDM@ IJCAI*, p. 39, 2013.

[Jame 23] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor. *An Introduction to Statistical Learning with Applications in Python. Springer Texts in Statistics*, Springer, Cham, 2023.

[Mass 24] K. Massow *et al.* "Crowd-Based Road Surface Assessment Using Smartphones on Bicycles". In: *Proc. of International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA 2024)*, 2024.

[Open 26a] OpenStreetMap Wiki Contributors. "Erklärung des Surface Tags". https://wiki.openstreetmap.org/wiki/Key:surface, 2026. Besucht am: 24. April 2026.

[Open 26b] OpenStreetMap Wiki Contributors. "Erklärung des Surface Tags". https://wiki.openstreetmap.org/wiki/Key:smoothness, 2026. Besucht am: 24. April 2026.

[Open 26c] OpenStreetMap Wiki Contributors. "Liste von OSM-basierten Android-Services". https://wiki.openstreetmap.org/wiki/Android, 2026. Besucht am: 24. April 2026.

[Open 26d] OpenStreetMap Wiki Contributors. "Liste von OSM-basierten Online-Routern". https://wiki.openstreetmap.org/wiki/Routing/online_routers, 2026. Besucht am: 24. April 2026.

[Rany 22] E. Ranyal, A. Sadhu, and K. Jain. "Road Condition Monitoring Using Smart Sensing and Artificial Intelligence: A Review". *Sensors*, Vol. 22, No. 8, 2022.

[Satt 18] S. Sattar, S. Li, and M. Chapman. "Road Surface Monitoring Using Smartphone Sensors: A Review". *Sensors*, Vol. 18, No. 3845, 2018.

[Saye 25] M. W. Sayers and S. M. Karamihas. "The Little Book of Profiling: Basic Information about Measuring and Interpreting Road Profiles". Tech. Rep., University of Michigan Transportation Research Institute, Ann Arbor, MI, September 2025. Second Edition.

[Saye 95] M. W. Sayers. "On the calculation of international roughness index from longitudinal road profile". *Transportation Research Record*, No. 1501, 1995.

[Scha 21] K. Schäfer and F. Quint. "Artificial Intellgence - Application in Life Sciences and Beyond. The Upper Rhine Artificial Intelligence Symposium UR-AI 2021". *CoRR*, Vol. abs/2112.05657, 2021.

[Scik 26] Scikit-learn developers. "Metrics and scoring: quantifying the quality of predictions". https://scikit-learn.org/stable/modules/model_evaluation.html, 2026. [Besucht am: 28. April 2026].

[Sera 16] F. Seraj, B. J. Van Der Zwaag, A. Dilo, T. Luarasi, and P. Havinga. "RoADS: A Road Pavement Monitoring System for Anomaly Detection Using Smart Phones". In: M. Atzmueller, A. Chin, F. Janssen, I. Schweizer, and C. Trattner, Eds., *Big Data Analytics in the Social and Ubiquitous Context*, pp. 128–146, Springer International Publishing, Cham, 2016. Series Title: Lecture Notes in Computer Science.

[Shub 24] Shubham and et al. "Enhanced Road Surface Classification using CNN and Random Forest Models". In: *Proceedings of 2nd International Conference on Sustainable Computing and Smart Systems*, 2024.

[STMi 23] STMicroelectronics. *LSM6DSO: always-on 3-axis accelerometer and 3-axis gyroscope*. 2023.

[Tark 25] S. Tarkiainen and J. Ikonen. "Effects of Sensor and Environmental Factors on Smartphone-Based Bicycle Road Monitoring". In: *2025 International Conference on Computer Systems and Technologies (CompSysTech)*, pp. 1–8, 2025.

[Varo 20] B. Varona, A. Monteserin, and A. Teyseyre. "A deep learning approach to automatic road surface monitoring and pothole detection". *Personal and Ubiquitous Computing*, Vol. 24, No. 4, pp. 519–534, 2020.

[Wu 20] S. Wu and A. Hadachi. "Road Surface Recognition Based on DeepSense Neural Network using Accelerometer Data". In: *IEEE Intelligent Vehicles Symposium*, 2020.

[Yili 26] B. Yiliguoqi and et al. "Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM". *Scientific Reports*, Vol. 16, No. 4409, 2026.

Glossar

library A suite of reusable code inside of a programming language for software development. i

shell Terminal of a Linux/Unix system for entering commands. i